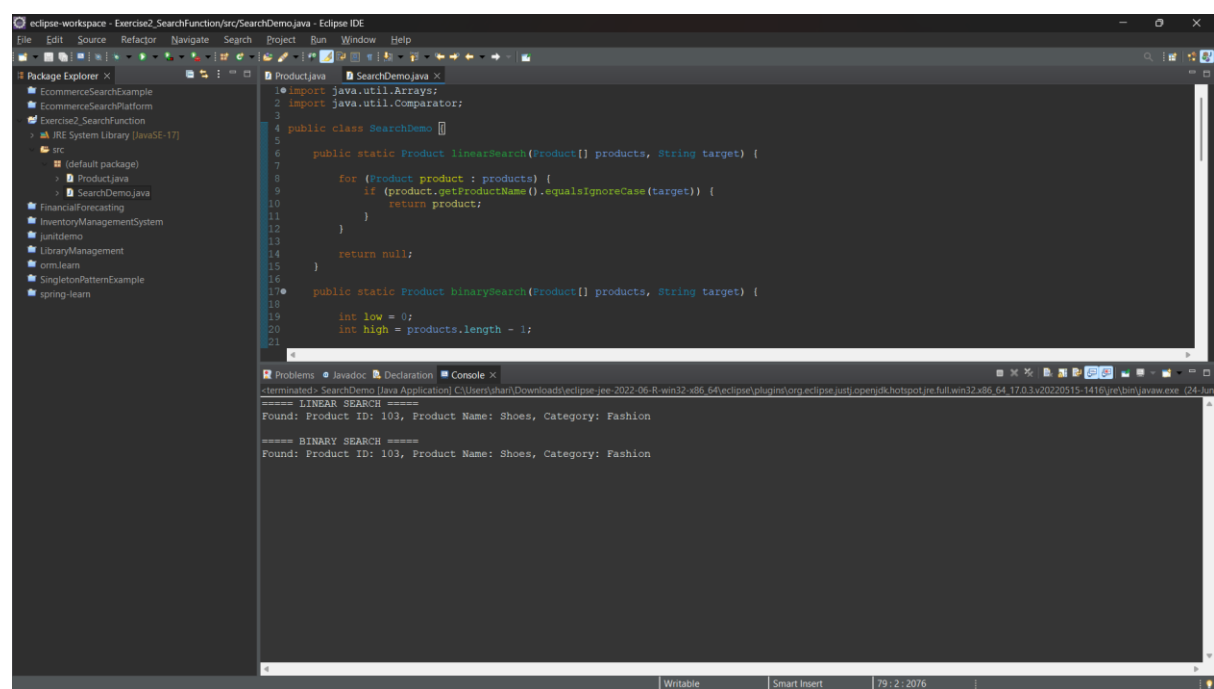


Exercise 2: E-commerce Platform Search Function

In this exercise, the search functionality of an e-commerce platform was implemented using Linear Search and Binary Search algorithms. A Product class was created with attributes such as productId, productName, and category to represent product information. Linear Search was performed on an unsorted array of products by checking each element sequentially until the required product was found. Binary Search was implemented on a sorted array of products, repeatedly dividing the search space into halves to locate the target product efficiently. Big O notation was used to analyze the performance of both algorithms. Linear Search has a time complexity of $O(n)$ in the average and worst cases, whereas Binary Search has a time complexity of $O(\log n)$, making it significantly faster for large datasets. Therefore, Binary Search is more suitable for an e-commerce platform with a large product catalog, provided that the data is maintained in sorted order.

OUTPUT



The screenshot displays the Eclipse IDE interface. The Package Explorer on the left shows the project structure, including the 'src' folder containing 'Product.java' and 'SearchDemo.java'. The main editor window shows the 'SearchDemo.java' file with the following code:

```
1 import java.util.Arrays;
2 import java.util.Comparator;
3
4 public class SearchDemo {
5
6     public static Product linearSearch(Product[] products, String target) {
7         for (Product product : products) {
8             if (product.getProductName().equalsIgnoreCase(target)) {
9                 return product;
10            }
11        }
12        return null;
13    }
14
15     public static Product binarySearch(Product[] products, String target) {
16         // Implementation of Binary Search
17         int low = 0;
18         int high = products.length - 1;
19     }
20 }
```

The Console window at the bottom shows the output of the program:

```
===== LINEAR SEARCH =====
Found: Product ID: 103, Product Name: Shoes, Category: Fashion
===== BINARY SEARCH =====
Found: Product ID: 103, Product Name: Shoes, Category: Fashion
```

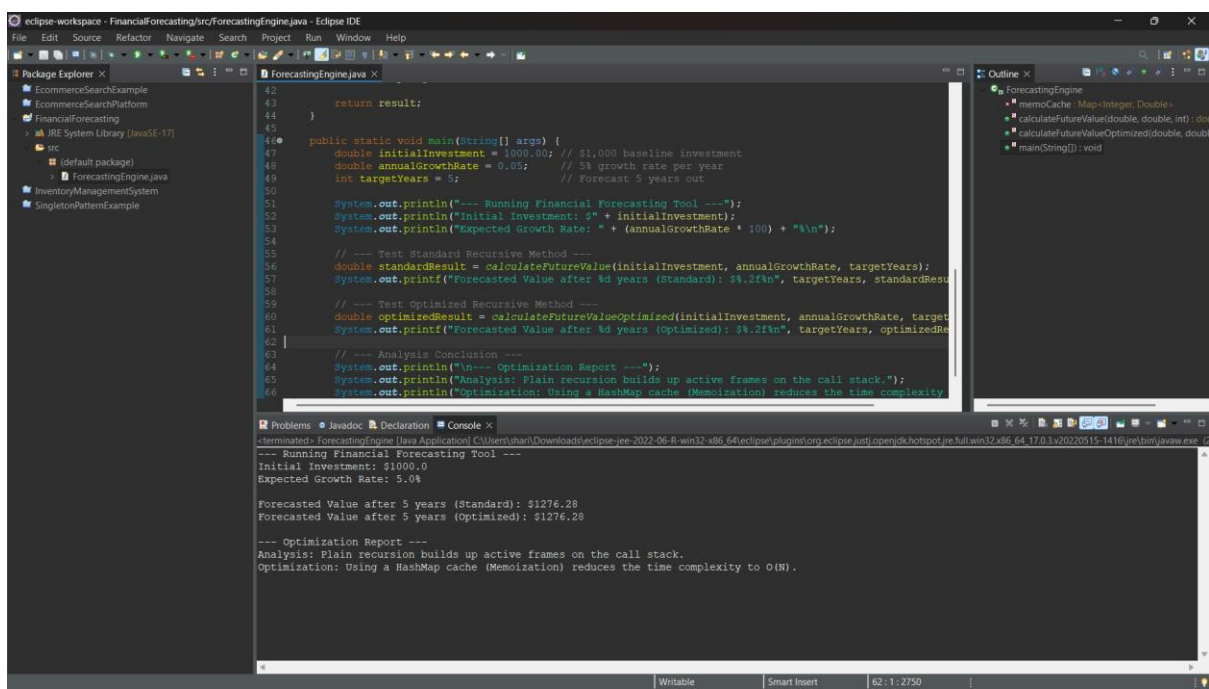
Exercise 7: Financial Forecasting

Recursion: Recursion is a programming technique where a method solves a problem by calling a smaller version of itself. Every recursive function must have a *Base Case* (to stop the loop) and a *Recursive Case* (to break the problem down).

Time Complexity:

- Unoptimized: Each recursive step branches into subsequent calls, which can cause an exponential time complexity of $O(2^N)$ or linear call space stack depths of $O(N)$.
- Optimized: By saving previously computed years into a cache (a technique called Memoization), we can eliminate recalculations, reducing the time complexity to a clean $O(N)$.

OUTPUT



```
42
43     return result;
44 }
45
46 public static void main(String[] args) {
47     double initialInvestment = 1000.00; // $1,000 baseline investment
48     double annualGrowthRate = 0.05;    // 5% growth rate per year
49     int targetYears = 5;               // Forecast 5 years out
50
51     System.out.println("--- Running Financial Forecasting Tool ---");
52     System.out.println("Initial Investment: $" + initialInvestment);
53     System.out.println("Expected Growth Rate: " + (annualGrowthRate * 100) + "%\n");
54
55     // --- Test Standard Recursive Method ---
56     double standardResult = calculateFutureValue(initialInvestment, annualGrowthRate, targetYears);
57     System.out.printf("Forecasted Value after %d years (Standard): %.2f\n", targetYears, standardResult);
58
59     // --- Test Optimized Recursive Method ---
60     double optimizedResult = calculateFutureValueOptimized(initialInvestment, annualGrowthRate, targetYears);
61     System.out.printf("Forecasted Value after %d years (Optimized): %.2f\n", targetYears, optimizedResult);
62
63     // --- Analysis Conclusion ---
64     System.out.println("\n--- Optimization Report ---");
65     System.out.println("Analysis: Plain recursion builds up active frames on the call stack.");
66     System.out.println("Optimization: Using a HashMap cache (Memoization) reduces the time complexity to O(N).");
67 }
```

```
--- Running Financial Forecasting Tool ---
Initial Investment: $1000.0
Expected Growth Rate: 5.0%

Forecasted Value after 5 years (Standard): $1276.28
Forecasted Value after 5 years (Optimized): $1276.28

--- Optimization Report ---
Analysis: Plain recursion builds up active frames on the call stack.
Optimization: Using a HashMap cache (Memoization) reduces the time complexity to O(N).
```